GRADLE

CHEAT SHEETS

A Complete Reference for Gradle 9 Builds

Wrapper & CLI · Kotlin DSL · Tasks · Providers · Dependencies · Version Catalogs · Platforms & BOMs · Toolchains · Testing · Multi-Project · Convention Plugins · Build Cache · Configuration Cache · Publishing · CI

27 REFERENCE SHEETS

CONTENTS

INTRODUCTION & SETUP

01 Introduction & How Gradle Works
02 Installing Gradle & the Wrapper
03 Project Structure & gradle init
04 Running Tasks from the CLI

BUILD SCRIPTS & TASKS

05 Kotlin DSL vs Groovy DSL
06 Applying Plugins
07 Tasks: Registering & Configuring
08 Properties & Providers

DEPENDENCIES

09 Declaring Dependencies & Configurations
10 Repositories
11 Version Catalogs
12 Conflicts, Constraints & Platforms

JAVA & JVM BUILDS

13 Java Plugin & Source Sets
14 Java Toolchains
15 Application Plugin & JARs
16 Testing

MULTI-PROJECT BUILDS

17 Multi-Project Builds
18 Convention Plugins (build-logic)
19 Composite Builds

PERFORMANCE & CUSTOM LOGIC

20 Build Cache & Configuration Cache
21 Daemon & Build Tuning
22 Writing Custom Tasks
23 Publishing Libraries

INTEGRATION & TROUBLESHOOTING

24 Spring Boot, Kotlin & Android Plugins
25 CI & Troubleshooting

QUICK REFERENCE

26 Quick Reference: CLI & gradle.properties
27 Quick Reference: Build Script Snippets

GRADLE

Table of Contents

27 sheets · syntax · examples · notes

INTRODUCTION & SETUP 01 – 04

- 01 Introduction & How Gr…
- 02 Installing Gradle & the …
- 03 Project Structure & gr…
- 04 Running Tasks from th…

BUILD SCRIPTS & TASKS 05 – 08

- 05 Kotlin DSL vs Groovy D…
- 06 Applying Plugins
- 07 Tasks: Registering & C…
- 08 Properties & Providers

DEPENDENCIES 09 – 12

- 09 Declaring Dependenci…
- 10 Repositories
- 11 Version Catalogs
- 12 Conflicts, Constraints …

JAVA & JVM BUILDS 13 – 16

- 13 Java Plugin & Source S…
- 14 Java Toolchains
- 15 Application Plugin & J…
- 16 Testing

MULTI-PROJECT BUILDS 17 – 19

- 17 Multi-Project Builds
- 18 Convention Plugins (b…
- 19 Composite Builds

PERFORMANCE & CUSTOM LOGIC 20 – 23

- 20 Build Cache & Configu…
- 21 Daemon & Build Tuning
- 22 Writing Custom Tasks
- 23 Publishing Libraries

INTEGRATION & TROUBLESHOOTING 24 – 25

- 24 Spring Boot, Kotlin & A…
- 25 CI & Troubleshooting

QUICK REFERENCE 26 – 27

- 26 Quick Reference: CLI &…
- 27 Quick Reference: Build …

Sheet 01

Introduction & How Gradle Works

build tool · tasks & task graph · init / config / execution phases · daemon · incremental builds · Gradle vs Maven

WHAT GRADLE IS

Gradle is a build automation tool for Java, Kotlin, Groovy, Scala, Android and C++. A build is a set of **projects**, each with **tasks** (compile, test, jar). Gradle wires tasks into a **directed acyclic graph** and runs only what changed, using up-to-date checks, a build cache and a long-lived **daemon**. Build scripts are real code written in the Kotlin DSL (`build.gradle.kts`) or the Groovy DSL (`build.gradle`).

GOOD TO KNOW

- Created in 2008; maintained by Gradle Inc. Current line: **9.x** (9.8, Sept 2026)
- Default build tool for Android; an option in Spring Initializr and IntelliJ
- Gradle 9 runs on **JVM 17+**; toolchains can still compile for Java 8
- `gradle init` generates Kotlin DSL by default

THE SMALLEST USEFUL BUILD

```
// build.gradle.kts
plugins { java }                // adds compileJava, test, jar, build…

tasks.register("hello") {       // a custom task, created lazily
    group = "demo"
    description = "Prints a greeting"
    doLast { println("Hello from Gradle!") }   // execution phase
}

// $ ./gradlew hello
// > Task :hello
// Hello from Gradle!
// BUILD SUCCESSFUL in 1s
```

CODE EXPLAINED

- → `plugins { java }` — applies the core Java plugin and its task set
- → `tasks.register` — declares a task; it is only configured if it will run
- → `group` / `description` — shown by `./gradlew tasks`
- → `doLast { }` — the action; code outside it runs at configuration time
- → `./gradlew` — the wrapper script; downloads the pinned Gradle version

IN GROOVY DSL

- → `build.gradle` — same script, Groovy file name
- → `plugins { java }` → `plugins { id 'java' }`
- → `println("…")` → `println '…'` — `'…'` is literal, `"…"` interpolates

JAVA VS KOTLIN PROJECTS

- → Java — `plugins { java }` : `compileJava` , sources in `src/main/java`
- → Kotlin — `kotlin("jvm") version "2.2.20"` : `compileKotlin` , `src/main/kotlin` ; applies `java` too
- → Task code (`register` , `doLast`) is identical in both

BUILD LIFECYCLE

PHASE	WHAT HAPPENS
Initialization	Reads `settings.gradle.kts`, decides which projects are in the build
Configuration	Runs each `build.gradle.kts`, creates and configures the needed tasks
Execution	Runs the requested tasks and their dependencies in graph order

CORE TERMS

TERM	MEANING
project	One buildable unit (one `build.gradle.kts`)
task	One unit of work: compile, copy, test
plugin	Adds tasks, conventions and extensions
configuration	A named group of dependencies
daemon	Background JVM reused between builds

GRADLE VS MAVEN

	GRADLE	MAVEN
Build file	Kotlin/Groovy code	pom.xml
Model	Task graph	Fixed lifecycle phases
Speed	Incremental + cache + daemon	Mostly full rebuilds
Custom logic	Tasks & plugins in code	Mojos / plugins

⚠ COMMON MISTAKES

- Putting work outside `doLast { }` — it then runs on **every** build during configuration, even when the task is not requested
- Running a globally installed `gradle` instead of `./gradlew` — you get a different Gradle version than the project expects
- Thinking Gradle needs Java 17 for your code — only the daemon does; set a **toolchain** to target older Java

✓ TIPS & BEST PRACTICES

- Run `./gradlew tasks` in any unfamiliar project to see what it can do
- Add `--scan` to a build for a shareable web report of what ran and why
- Prefer the Kotlin DSL for new builds: compile-time checks and IDE completion

QUICK SUMMARY

- `project` + `task` — the two building blocks of every build
- `init → configure → execute` — the three build phases
- `./gradlew <task>` — run a task through the wrapper
- `doLast { }` — where task work belongs

Sheet 02

Installing Gradle & the Wrapper

gradlew / gradlew.bat · gradle-wrapper.properties · gradle wrapper -- gradle-version · distributionUrl · SDKMAN / Homebrew · Java 17+

THE WRAPPER

The **Gradle Wrapper** (`gradlew`, `gradlew.bat` and the `gradle/wrapper/` folder) pins the exact Gradle version a project uses and downloads it on first run. Everyone, including CI, builds with the same version without installing Gradle. You only need a local Gradle install to create a wrapper for a brand-new project; after that, always run `./gradlew`.

REQUIREMENTS

- A JDK **17+** on `PATH` or in `JAVA_HOME` to run Gradle 9
- Commit all four wrapper files to version control
- Distributions are cached in `~/.gradle/wrapper/dists`
- `-bin` = runtime only; `-all` adds sources and docs for IDE help

INSTALL & MANAGE THE WRAPPER

```
java -version                          # must report 17 or newer
# Install Gradle once (only needed to create new wrappers)
sdk install gradle 9.8.0               # SDKMAN (macOS / Linux)
brew install gradle                    # Homebrew
gradle --version                       # Gradle 9.8.0, JVM …
# Create or upgrade the wrapper in a project
gradle wrapper --gradle-version 9.8.0
./gradlew wrapper --gradle-version latest
./gradlew wrapper --distribution-type all
# Build through the wrapper from now on
./gradlew build                        # macOS / Linux
gradlew.bat build                      # Windows (cmd / PowerShell)
chmod +x gradlew                       # fix "permission denied"
```

CODE EXPLAINED

- → `sdk` / `brew` / `choco` — package managers for a local install
- → `gradle wrapper` — writes `gradlew`, `gradlew.bat` and `gradle/wrapper/*`
- → `--gradle-version latest` — resolves the newest stable release
- → `--distribution-type all` — downloads the larger `-all` ZIP
- → Run `wrapper` twice when upgrading: the second run updates the scripts with the new version

WRAPPER FILES (COMMIT ALL)

FILE	PURPOSE
`gradlew`	Unix shell launcher
`gradlew.bat`	Windows launcher
`gradle/wrapper/gradle-wrapper.jar`	Small bootstrap that downloads Gradle
`gradle/wrapper/gradle-wrapper.properties`	Which version to download and from where

GRADLE-WRAPPER.PROPERTIES KEYS

KEY	EXAMPLE / MEANING
`distributionUrl`	`…/distributions/gradle-9.8.0-bin.zip`
`distributionSha256Sum`	Verifies the download; set it for supply-chain safety
`networkTimeout`	Download timeout in ms (default `10000`)
`validateDistributionUrl`	`true` — fail early on a bad URL
`distributionBase / Path`	Where unpacked versions live (`GRADLE_USER_HOME`)

⚠ COMMON MISTAKES

- Not committing `gradle-wrapper.jar` (a `*.jar` ignore rule hides it) — fresh clones can't build
- Editing `distributionUrl` by hand — re-run `./gradlew wrapper` so the scripts update too
- Running on Java 11 — Gradle 9 refuses to start; point `JAVA_HOME` at a 17+ JDK

✓ TIPS & BEST PRACTICES

- Use `--gradle-distribution-sha256-sum` when upgrading so the checksum is written for you
- Add `gradle-wrapper.jar` validation to CI with `gradle/actions/wrapper-validation`

QUICK SUMMARY

- `./gradlew` — always build through the wrapper
- `gradle wrapper --gradle-version X` — pin or upgrade the version
- `gradle-wrapper.properties` — source of truth for the version
- `JDK 17+` — needed to run Gradle 9

Project Structure & gradle init

ANATOMY OF A BUILD

A Gradle build has one **settings file** at the root that names the build and lists its subprojects, plus one **build script** per project. Shared flags go in `gradle.properties` and dependency versions in the **version catalog** `gradle/libs.versions.toml`. `gradle init` scaffolds all of this, including the wrapper, using the Maven-style `src/main` / `src/test` layout.

NOTES

- `gradle init` in a folder with a `pom.xml` offers to convert the Maven build
- Generated apps put code in an `app/` subproject, not the root
- `.gradle/` and every `build/` folder are generated: add them to `.gitignore`
- `buildSrc/` or `build-logic/` holds shared build logic (sheet 18)

TYPICAL LAYOUT

```
my-app/
├── settings.gradle.kts       # build name + included subprojects
├── gradle.properties         # Gradle flags (jvmargs, caching…)
├── gradlew  gradlew.bat      # wrapper scripts (commit)
├── gradle/
│   ├── wrapper/              # wrapper jar + properties (commit)
│   └── libs.versions.toml    # version catalog
├── app/
│   ├── build.gradle.kts      # this subproject's build script
│   └── src/
│       ├── main/java/        # production code
│       ├── main/resources/   # config files, templates
│       ├── test/java/        # tests
│       └── test/resources/
├── .gradle/                  # local caches (ignore)
└── app/build/                # outputs: classes, jars, reports
```

KEY FILES

- → `settings.gradle.kts` — `rootProject.name = "my-app"` and `include("app")`
- → `build.gradle.kts` — plugins, dependencies and task config for one project
- → `gradle.properties` — `org.gradle.caching=true` and other switches
- → `libs.versions.toml` — one place for every dependency version
- → `build/libs/` — where `./gradlew jar` puts the JAR

GRADLE INIT --TYPE

TYPE	GENERATES
`basic`	Empty build with wrapper and settings
`java-application`	App with `application` plugin + JUnit
`java-library`	Library with `java-library` plugin
`kotlin-application`	Kotlin JVM app
`kotlin-library`	Kotlin JVM library
`java-gradle-plugin`	A Gradle plugin project
`pom`	Converts an existing Maven build

GRADLE INIT OPTIONS

OPTION	EXAMPLE
`--dsl`	`kotlin` (default) or `groovy`
`--test-framework`	`junit-jupiter`, `testng`, `spock`, `kotlintest`
`--project-name`	`my-app`
`--package`	`com.example.app`
`--java-version`	21 (writes a toolchain)
`--split-project`	App + library subprojects with convention plugins
`--use-defaults`	Skip all prompts

⚠ COMMON MISTAKES

- Running `gradle init` where a build already exists — it stops (or offers to convert a `pom.xml`); start in an empty folder
- Committing `build/` or `.gradle/` — huge diffs and stale outputs on other machines
- Forgetting to `include()` a new subproject in settings — Gradle silently ignores its folder

✓ TIPS & BEST PRACTICES

- Script it: `gradle init --type java-application --dsl kotlin --use-defaults`
- Keep `rootProject.name` explicit; otherwise it is the folder name, which varies on CI
- Use `./gradlew projects` to print the project tree Gradle sees

QUICK SUMMARY

- `settings.gradle.kts` — which projects are in the build
- `build.gradle.kts` — how one project is built
- `gradle/libs.versions.toml` — dependency versions
- `gradle init` — scaffold a new build

Running Tasks from the CLI

./gradlew build · tasks --all · task abbreviations · -x exclude · --continue · --dry-run · :sub:task paths · --rerun-tasks

RUNNING TASKS

You run a build by naming one or more **tasks**; Gradle adds their dependencies and runs each task once, in graph order. Task names can be abbreviated by camelCase (`cTJ` for `compileTestJava`) and qualified with a **project path** (`:app:test`). Run without a path, a task runs in the current project **and** every subproject that has it.

TASK OUTCOMES

- `UP-TO-DATE` — inputs and outputs unchanged, skipped
- `FROM-CACHE` — outputs restored from the build cache
- `NO-SOURCE` — no input files, nothing to do
- `SKIPPED` — excluded with `-x` or `onlyIf` false
- No label — the task actually executed

EVERYDAY COMMANDS

```
./gradlew build                 # compile, test, jar — the works
./gradlew tasks                 # tasks in named groups
./gradlew tasks --all           # every task, including ungrouped
./gradlew help --task test      # details and options for one task
./gradlew build -x test         # exclude a task
./gradlew test --continue       # keep going after a failure
./gradlew build --dry-run       # list what would run (-m)
./gradlew :app:run              # task in one subproject
./gradlew test --tests '*UserTest'   # task option
./gradlew test --rerun          # force just this task to run
./gradlew build --rerun-tasks   # ignore up-to-date for all tasks
```

NOTES

- Task names are case-sensitive; abbreviations must be unambiguous
- Options after a task name (`--tests`) belong to that task
- `-x test` also skips tasks only `test` needed
- `:app:run` is an absolute path; `app:run` is relative to the current dir
- `--rerun` (one task) is much cheaper than `--rerun-tasks` (everything)

COMMON TASKS (JAVA PLUGIN)

TASK	DOES
`clean`	Deletes `build/`
`assemble`	Builds all outputs, no tests
`check`	Runs tests and verification
`build`	`assemble` + `check`
`test`	Runs unit tests
`jar`	Packages main classes
`dependencies`	Prints dependency trees
`projects`	Prints the project tree

USEFUL FLAGS

FLAG	EFFECT
`-q / -i / -d`	Quiet / info / debug logging
`--scan`	Publish a build scan
`--offline`	Use only cached dependencies
`--parallel`	Run projects in parallel
`-Pname=value`	Set a project property
`-Dname=value`	Set a JVM system property
`--warning-mode all`	Show every deprecation

TASK PATHS

PATH	RUNS
`test`	`test` in this and all subprojects
`:test`	Only the root project's `test`
`:app:test`	Only `app`'s `test`
`:lib:core:jar`	Nested project `lib/core`
`-p app test`	Run from the `app` directory

⚠ COMMON MISTAKES

- Expecting `./gradlew test` to run only the root project — it runs `test` in **every** subproject; use `:test` for root only
- Using `clean build` out of habit — `clean` throws away incremental work and makes every build a full rebuild
- Using the removed `-b build.gradle.kts` option — Gradle 9 dropped `-b` / `-c` ; use `-p <dir>`

✓ TIPS & BEST PRACTICES

- `./gradlew help --task <name>` lists a task's type, path and options
- Combine `--dry-run` with a task list to check ordering before a slow build
- Use `--continue` in CI to collect every failing test module in one run

QUICK SUMMARY

- `./gradlew <task>` — run a task plus its dependencies
- `-x <task>` — exclude a task
- `:project:task` — target one project
- `tasks --all` — discover what exists

Kotlin DSL vs Groovy DSL

TWO DSLS, ONE API

Both DSLs drive the same Gradle API, so every concept on these sheets applies to both. The **Kotlin DSL** (`.gradle.kts`) is statically typed: the IDE completes and checks it, and mistakes fail at script compile time. The **Groovy DSL** (`.gradle`) is dynamic and terser but errors show up only when the line runs. Kotlin is the default for new builds; you can mix both file types in one build.

NOTES

- Kotlin DSL scripts compile once, then are cached
- Gradle 9 embeds Kotlin **2.2** and Groovy **4**
- Groovy space-assignment (`version '1.0'`) is deprecated; use `=`
- Type-safe accessors (`libs` , `java { }`) exist only for plugins in `plugins { }`

BUILD.GRADLE.KTS (KOTLIN)

```
plugins {
    id("application")
    id("com.gradleup.shadow") version "9.0.0"
}
group = "com.example"
version = "1.0.0"
repositories { mavenCentral() }
dependencies {
    implementation("com.google.guava:guava:33.4.0-jre")
    testImplementation(libs.junit.jupiter)
    testRuntimeOnly(libs.junit.launcher)
}
application { mainClass = "com.example.App" }
tasks.named<Test>("test") {
    useJUnitPlatform()
    maxHeapSize = "1g"
}
```

BUILD.GRADLE (GROOVY)

```
plugins {
    id 'application'
    id 'com.gradleup.shadow' version '9.0.0'
}
group = 'com.example'
version = '1.0.0'
repositories { mavenCentral() }
dependencies {
    implementation 'com.google.guava:guava:33.4.0-jre'
    testImplementation libs.junit.jupiter
    testRuntimeOnly libs.junit.launcher
}
application { mainClass = 'com.example.App' }
tasks.named('test', Test) {
    useJUnitPlatform()
    maxHeapSize = '1g'
}
```

SYNTAX DIFFERENCES

KOTLIN DSL	GROOVY DSL	NOTE
`"text"`	`'text' or "text"`	Kotlin strings are always double-quoted
`id("java")`	`id 'java'`	Kotlin requires parentheses on calls
`version = "1.0"`	`version = '1.0'`	Use = in both
`tasks.named<Test>("test")`	`tasks.named('test', Test)`	Typed lookup
`` `java-library` ``	`id 'java-library'`	Backticks for ids with a dash

MIGRATING GROOVY → KOTLIN

STEP	DO
1	Rename `settings.gradle`, then each build file, to `.kts`
2	Change `'` to `"`; add `()` to method calls
3	Replace `ext` with `extra` or the catalog
4	Move `apply plugin:` into `plugins { }`

⚠ COMMON MISTAKES

- Single quotes in `.kts` (`id('java')`) — that is a Kotlin `Char` literal, so the script fails to compile
- Using `tasks.test { }` -style accessors for a plugin applied with `apply(plugin = …)` — accessors only exist for `plugins { }`
- Using `"test"() { }` task syntax from old Kotlin DSL samples — removed in Gradle 9; use `tasks.named("test")`

✓ TIPS & BEST PRACTICES

- Convert one file at a time; Groovy and Kotlin scripts can live side by side
- After editing a `.kts` file, sync the IDE so new accessors appear
- Run `./gradlew help` after a migration to catch compile errors fast

QUICK SUMMARY

- `.gradle.kts` — typed, IDE-checked, default
- `.gradle` — dynamic Groovy, terser
- `"…"` + `()` — the two main Kotlin syntax rules
- `tasks.named<T>()` — typed task access in Kotlin

Applying Plugins

PLUGINS

Almost everything a build does comes from **plugins**: they add tasks, conventions (like `src/main/java`) and configuration blocks (extensions). **Core plugins** ship with Gradle and need no version. **Community plugins** come from the Gradle Plugin Portal or a Maven repository and need a version. Declare them in the `plugins { }` block at the top of a build script so Gradle can resolve them early and generate type-safe accessors.

NOTES

- `plugins { }` must come first; only `buildscript { }` may precede it
- `plugins { }` cannot use variables; use `alias(libs.plugins…)` for versions
- Declare a version once (root, `apply false`), then apply by id in subprojects
- Legacy forms to replace: `apply(plugin = …)` and `buildscript { classpath(…) }`

DECLARING PLUGINS

```
// settings.gradle.kts — where plugins come from
pluginManagement {
    repositories { gradlePluginPortal(); mavenCentral() }
}
// root build.gradle.kts — declare versions, apply nowhere
plugins {
    id("com.gradleup.shadow") version "9.0.0" apply false
}
// app/build.gradle.kts
plugins {
    `java-library`                  // core id with a dash → backticks
    application                     // core plugin, no version
    id("com.gradleup.shadow")       // version comes from the root
    alias(libs.plugins.spring.boot) // id + version from the catalog
}
```

CODE EXPLAINED

- → `pluginManagement` — plugin repositories for the whole build
- → `apply false` — put the plugin on the classpath without applying it
- → Backticks — Kotlin accessor form of a core id with a dash
- → `alias(libs.plugins.x)` — id and version from `libs.versions.toml`

IN GROOVY DSL

- → `` `java-library` `` → `id 'java-library'` — no backticks needed
- → `id("x") version "9.0.0" apply false` → `id 'x' version '9.0.0' apply false`
- → `alias(libs.plugins.x)` — same in both DSLs

JAVA VS KOTLIN PROJECTS

- → Java — core plugins `java`, `java-library`, `application`; no version
- → Kotlin — `kotlin("jvm")` = `id("org.jetbrains.kotlin.jvm")`; not core, needs a version
- → `` `kotlin-dsl` `` is for writing build logic, not for Kotlin apps

CORE PLUGINS

PLUGIN	ADDS
`java`	Compile, test, jar for Java
`java-library`	`java` + the `api` configuration
`application`	`run`, start scripts, `distZip`
`java-platform`	Publish a BOM of versions
`maven-publish`	Publish to Maven repos
`jacoco`	Code coverage reports

COMMON COMMUNITY PLUGINS

PLUGIN ID	FOR
`org.springframework.boot`	Spring Boot apps
`org.jetbrains.kotlin.jvm`	Kotlin — `kotlin("jvm")`
`com.android.application`	Android apps (AGP)
`com.gradleup.shadow`	Fat / shaded JARs
`com.diffplug.spotless`	Code formatting
`org.graalvm.buildtools.native`	GraalVM native images

⚠ COMMON MISTAKES

- Giving the same plugin a version in two subprojects — Gradle fails with a conflict; declare it once in the root with `apply false`
- Adding a version to a core plugin (`id("java") version …`) — core plugins are versioned with Gradle and reject a version
- Using `com.github.johnrengelman.shadow` — that id is abandoned; the plugin moved to `com.gradleup.shadow`

✓ TIPS & BEST PRACTICES

- Keep plugin versions in `[plugins]` of the version catalog (sheet 11)
- Run `./gradlew buildEnvironment` to see which plugin versions resolved
- Prefer `plugins { }` over `buildscript { }`: faster and gives accessors

QUICK SUMMARY

- `plugins { id("x") version "v" }` — apply a community plugin
- `plugins { java }` — apply a core plugin
- `apply false` — declare once, apply later
- `pluginManagement` — where plugins come from

Tasks: Registering & Configuring

tasks.register · tasks.named · doFirst / doLast · dependsOn · finalizedBy · mustRunAfter · group & description · lazy vs eager

TASKS

A **task** has inputs, outputs and one or more **actions** (`doFirst`, `doLast`). Register tasks with `tasks.register`, which is **lazy**: the configuration block runs only if the task ends up in the graph. Change existing tasks with `tasks.named` or `tasks.withType<T>().configureEach`. Wiring an output to another task's input, or using `dependsOn`, tells Gradle the order.

NOTES

- Typed tasks: `tasks.register<Copy>("x")` gives `from`, `into` …
- Declare `inputs` / `outputs` so the task can be `UP-TO-DATE`
- Inside actions, don't touch `project` — capture values first (config cache)

REGISTER, CONFIGURE, ORDER

```
val generateInfo = tasks.register("generateInfo") {
    group = "build"
    description = "Writes build-info.txt"
    val out = layout.buildDirectory.file("build-info.txt")
    val ver = project.version.toString()      // captured at config time
    inputs.property("version", ver)
    outputs.file(out)
    doLast { out.get().asFile.writeText("version=$ver\n") }
}
tasks.named("processResources") { dependsOn(generateInfo) }
tasks.register<Copy>("copyDocs") {             // typed task
    from("docs"); into(layout.buildDirectory.dir("site"))
}
tasks.withType<JavaCompile>().configureEach { options.encoding = "UTF-8" }
tasks.named<Test>("test") { finalizedBy("jacocoTestReport") }
```

CODE EXPLAINED

- → `tasks.register` — returns a `TaskProvider`; configured lazily
- → `inputs` / `outputs` — enable up-to-date checks and caching
- → `doLast` — the action that runs at execution time
- → `dependsOn(generateInfo)` — run it before `processResources`
- → `register<Copy>` — a built-in task type with its own DSL
- → `withType<T>().configureEach` — lazily configure every task of a type

IN GROOVY DSL

- → `register<Copy>("x")` → `register('x', Copy)`
- → `withType<JavaCompile>()` → `withType(JavaCompile)`
- → `val ver = …` → `def ver = …`; `"$ver"` interpolates, `'…'` doesn't

JAVA VS KOTLIN PROJECTS

- → Java — `withType<JavaCompile>`: `options.encoding`, `options.compilerArgs`
- → Kotlin — `withType<KotlinCompile>`: `compilerOptions { }`; sources are always UTF-8
- → `processResources`, `test`, `jar` — same tasks in both

ORDERING & CONDITIONS

API	MEANING
`dependsOn(t)`	`t` must run first (and is pulled in)
`mustRunAfter(t)`	If both run, this runs after `t`
`shouldRunAfter(t)`	Soft version of `mustRunAfter`
`finalizedBy(t)`	Run `t` after this, even on failure
`onlyIf { … }`	Skip unless the condition is true

LAZY VS EAGER APIS

API	LAZY?
`tasks.register(…)`	✔
`tasks.named(…)`	✔
`withType<T>().configureEach`	✔
`tasks.create(…)`	✖ eager
`tasks.getByName(…)`	✖ eager
`withType<T> { … }`	✖ eager

BUILT-IN TASK TYPES

TYPE	USE
`Copy` / `Sync`	Copy files (Sync deletes extras)
`Delete`	Remove files
`Zip` / `Jar`	Create archives
`Exec` / `JavaExec`	Run a program / main class

⚠ COMMON MISTAKES

- Using `tasks.create` or `withType<T> { }` — eager APIs configure every task on every build and slow configuration
- Reading `project.something` inside `doLast` — breaks the configuration cache; capture values in a `val` outside the action
- Using `dependsOn` when you only need ordering — use `mustRunAfter` so you don't force the other task to run

✓ TIPS & BEST PRACTICES

- Pass a `TaskProvider` (not a name string) to `dependsOn` for safer wiring
- Give every custom task a `group` so it shows in `./gradlew tasks`
- Wire outputs to inputs (`from(otherTask)`) — dependencies are then inferred

QUICK SUMMARY

- `tasks.register` — create a task lazily
- `tasks.named` — configure an existing task
- `doLast { }` — task action
- `dependsOn` / `mustRunAfter` — ordering

Properties & Providers

LAZY CONFIGURATION

Modern Gradle APIs use **lazy properties**: a `Provider<T>` is a value that is computed later, and a `Property<T>` is a provider you can `set()`. Wiring providers together (`map`, `flatMap`, `orElse`) lets Gradle read values only at execution time and track which task produced them. Read build settings with `providers.gradleProperty(…)` and `providers.environmentVariable(…)` instead of `System.getenv`.

WHERE VALUES COME FROM

- `-Pname=value` on the command line (highest priority)
- `~/.gradle/gradle.properties` (user-wide, keep secrets here)
- `gradle.properties` in the project root
- `ORG_GRADLE_PROJECT_name` environment variable
- `-Dname=value` — JVM system property, not a project property

READING AND WIRING VALUES

```
// gradle.properties →  appEnv=dev
val appEnv = providers.gradleProperty("appEnv").orElse("dev")
val apiKey = providers.environmentVariable("API_KEY")
val gitSha = providers.exec { commandLine("git", "rev-parse", "HEAD") }
    .standardOutput.asText.map { it.take(7) }

tasks.register("showEnv") {
    val env = appEnv; val sha = gitSha   // capture providers, not project
    doLast { println("env=${env.get()} sha=${sha.get()}") }
}
tasks.named<Jar>("jar") {
    archiveBaseName.set("my-app")        // Property<String>.set()
    archiveVersion = appEnv.map { "1.0-$it" }  // lazy assignment
}
// ./gradlew showEnv -PappEnv=prod  →  env=prod sha=3f2a9c1
```

CODE EXPLAINED

- → `providers.gradleProperty` — reads `-P` / `gradle.properties` lazily
- → `.orElse("dev")` — fallback when the property is missing
- → `providers.exec` — runs a command; replaces the removed `project.exec`
- → `.map { }` — transform without reading the value yet
- → `.get()` — read the value; only inside task actions

IN GROOVY DSL

- → `val x = providers…` → `def x = providers…`
- → `named<Jar>("jar")` → `named('jar', Jar)`
- → `"${env.get()}"` and `archiveVersion = …` work the same

JAVA VS KOTLIN PROJECTS

- → Same `providers` / `Property` API in every project type
- → Java — `tasks.compileJava { options.release = 21 }`
- → Kotlin — `kotlin { compilerOptions { jvmTarget = JvmTarget.JVM_21 } }`

PROVIDER API

METHOD	DOES
`get()`	Value, or error if absent
`getOrElse(v)`	Value or v
`orElse(v)`	Provider with a fallback
`map { }`	Transform lazily
`flatMap { }`	Map to another provider
`zip(other) { a, b -> }`	Combine two providers

OLD HABIT → LAZY WAY

OLD	NEW
`project.findProperty("x")`	`providers.gradleProperty("x")`
`System.getenv("X")`	`providers.environmentVariable("X")`
`project.exec { }`	`providers.exec { }`
`buildDir`	`layout.buildDirectory`
`ext["x"] = …`	`extra` or a typed extension

⚠ COMMON MISTAKES

- Calling `.get()` at configuration time — forces the value early, may run a command on every build and defeats lazy wiring
- Putting secrets in the project's `gradle.properties` — it gets committed; use `~/.gradle/gradle.properties` or env vars
- Confusing `-D` with `-P` — `-D` sets a JVM system property, which `gradleProperty()` will not see

✓ TIPS & BEST PRACTICES

- Prefix with `systemProp.` in `gradle.properties` to set a system property
- In CI, pass secrets as `ORG_GRADLE_PROJECT_name` env vars
- Expose settings as `Property` fields on an extension for typed, lazy config

QUICK SUMMARY

- `Provider<T>` — read-only lazy value
- `Property<T>` — settable lazy value
- `providers.gradleProperty("x")` — read `-Px`
- `.map { }` — transform without resolving

Declaring Dependencies & Configurations

DEPENDENCIES

Dependencies are declared into **configurations**, named buckets that decide where a library is visible: at compile time, at runtime, in tests, or to consumers of your library. Use `implementation` by default. Use `api` (from `java-library`) only when a type from the dependency appears in your public API. Coordinates follow Maven's `group:name:version` format.

NOTES

- `api` exists only with the `java-library` plugin
- Gradle 9 needs `junit-platform-launcher` declared explicitly
- Prefer catalog entries (`libs.x`) over inline strings
- Inspect: `./gradlew :app:dependencies --configuration runtimeClasspath`

A DEPENDENCIES { } BLOCK

```
plugins { `java-library` }

dependencies {
    api("com.fasterxml.jackson.core:jackson-databind:2.19.0") // public API
    implementation("com.google.guava:guava:33.4.0-jre")       // internal
    implementation(project(":core"))                          // subproject
    implementation(libs.okhttp)                               // catalog
    compileOnly("org.projectlombok:lombok:1.18.38")           // no runtime
    annotationProcessor("org.projectlombok:lombok:1.18.38")
    runtimeOnly("org.postgresql:postgresql:42.7.7")           // runtime only
    testImplementation("org.junit.jupiter:junit-jupiter:5.13.4")
    testRuntimeOnly("org.junit.platform:junit-platform-launcher")
    implementation(files("libs/legacy.jar"))                  // local JAR
}
```

CODE EXPLAINED

- → `api` — consumers also get Jackson on their compile classpath
- → `implementation` — hidden from consumers; faster recompiles
- → `project(":core")` — depend on a subproject in the same build
- → `compileOnly` — provided at runtime by something else
- → `annotationProcessor` — processors run by `javac`
- → `files(…)` — a local JAR; prefer a repository when possible

IN GROOVY DSL

- → `implementation("g:a:v")` → `implementation 'g:a:v'`
- → `project(":core")` → `project(':core')` ; `libs.okhttp` unchanged
- → `` `java-library` `` → `id 'java-library'`

JAVA VS KOTLIN PROJECTS

- → Java — `annotationProcessor(…)` runs processors in javac
- → Kotlin — use `kapt(…)` (`kotlin("kapt")`) or `ksp(…)` instead
- → Kotlin — stdlib is added automatically; `testImplementation(kotlin("test"))`

CONFIGURATIONS

CONFIGURATION	COMPILE	RUNTIME	CONSUMERS
`api`	✔	✔	✔
`implementation`	✔	✔	runtime
`compileOnly`	✔	✖	✖
`compileOnlyApi`	✔	✖	compile
`runtimeOnly`	✖	✔	runtime
`testImplementation`	tests	tests	✖
`testRuntimeOnly`	✖	tests	✖

DEPENDENCY NOTATIONS

NOTATION	MEANING
`"g:a:1.2.3"`	Group, artifact, version
`"g:a:1.2.3:sources"`	With a classifier
`"g:a"`	Version from a platform / constraint
`project(":lib")`	Subproject
`libs.guava`	Version catalog entry
`platform("g:bom:v")`	Import a BOM (sheet 12)
`testFixtures(project(":core"))`	Test fixtures of a project

⚠ COMMON MISTAKES

- Using `compile` / `testCompile` from old tutorials — long removed; use `implementation`
- Making everything `api` — every change recompiles all downstream projects
- Leaving out the JUnit `junit-platform-launcher` — Gradle 9 test runs fail to start

✓ TIPS & BEST PRACTICES

- Ask Gradle why a library is present: `./gradlew dependencyInsight --dependency guava`
- Use the JUnit BOM (`platform(...)`) so all JUnit modules share one version
- Keep `annotationProcessor` in step with `compileOnly` for Lombok-style tools

QUICK SUMMARY

- `implementation` — the default choice
- `api` — only for types in your public API
- `compileOnly` / `runtimeOnly` — one side of the classpath
- `testImplementation` — test-only dependencies

Repositories

mavenCentral() · google() · maven { url } · mavenLocal() · credentials · dependencyResolutionManagement · content filtering

WHERE DEPENDENCIES COME FROM

A **repository** is a server or folder Gradle downloads modules from, checked in the order you declare them. Declare repositories once in `settings.gradle.kts` under `dependencyResolutionManagement` so every project uses the same list. Use **content filtering** so each group is looked up only where it lives: faster builds and less risk of dependency-confusion attacks.

NOTES

- `jcenter()` was removed in Gradle 9; use `mavenCentral()`
- Downloads are cached in `~/.gradle/caches/modules-2`
- Plugin repositories are separate: `pluginManagement { repositories { } }`
- `--offline` uses only the cache; `--refresh-dependencies` re-checks remotes

CENTRAL REPOSITORY SETUP

```
// settings.gradle.kts
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        mavenCentral()
        google()                                  // Android / AndroidX
        maven("https://repo.example.com/releases") {
            name = "company"            // → companyUsername / companyPassword
            credentials(PasswordCredentials::class)
            content { includeGroupByRegex("com\\.example.*") }
        }
        mavenLocal { content { includeGroup("com.example.dev") } }
    }
}
```

CODE EXPLAINED

- → `dependencyResolutionManagement` — repositories for every project
- → `FAIL_ON_PROJECT_REPOS` — error if a project declares its own
- → `maven(url)` — any Maven-layout repository (Nexus, Artifactory)
- → `credentials(PasswordCredentials::class)` — read from `<name>Username` / `<name>Password` properties
- → `content { }` — only look up matching groups here

IN GROOVY DSL

- → `settings.gradle` — same blocks
- → `maven("url") { … }` → `maven { url = '…'; name = 'company' }`
- → `credentials(PasswordCredentials::class)` → `credentials(PasswordCredentials)`

JAVA VS KOTLIN PROJECTS

- → Repositories are the same for Java and Kotlin projects
- → Kotlin — stdlib and kotlinx libraries resolve from `mavenCentral()`
- → Android / Kotlin Multiplatform builds also need `google()`

REPOSITORY SHORTCUTS

CALL	REPOSITORY
`mavenCentral()`	Maven Central — the default for JVM
`google()`	Google Maven (Android, AndroidX)
`gradlePluginPortal()`	Gradle plugins (and their dependencies)
`mavenLocal()`	`~/.m2/repository` — avoid except for local tests
`maven("https://…")`	Any Maven repository by URL

REPOSITORIESMODE

MODE	BEHAVIOR
`PREFER_PROJECT`	Default: project repos override settings
`PREFER_SETTINGS`	Settings win; project repos ignored
`FAIL_ON_PROJECT_REPOS`	Project repos cause an error

⚠ COMMON MISTAKES

- Putting `mavenLocal()` first — stale local builds shadow real releases and builds differ per machine
- Committing repository passwords in build files — use `~/.gradle/gradle.properties` or env vars
- Declaring repos in every subproject — they drift apart; centralize them in settings

✓ TIPS & BEST PRACTICES

- Pass secrets in CI as `ORG_GRADLE_PROJECT_companyPassword`
- Use `exclusiveContent { }` to force a group to come from one repo only
- Order repos by how often they hit: Central first, private last

QUICK SUMMARY

- `dependencyResolutionManagement` — one repo list for all
- `mavenCentral()` — the standard source
- `maven(url) { credentials(…) }` — private repos
- `content { includeGroup }` — scope lookups

Version Catalogs

ONE FILE FOR VERSIONS

A **version catalog** is a TOML file, `gradle/libs.versions.toml`, that names every dependency and plugin with its version. Gradle reads it automatically and generates a type-safe `libs` accessor, so build scripts say `libs.guava` instead of repeating coordinates. A catalog only **declares** names; it does not add anything to a project until you use an entry.

NAMING RULES

- `-`, `_` and `.` in keys become dots: `junit-jupiter` → `libs.junit.jupiter`
- Plugins are under `libs.plugins.*`, bundles under `libs.bundles.*`
- Versions: `libs.versions.kotlin.get()` gives the string

GRADLE/LIBS.VERSIONS.TOML

```
[versions]
kotlin = "2.2.20"
junit = "5.13.4"
springBoot = "4.1.0"

[libraries]
guava = "com.google.guava:guava:33.4.0-jre"
junit-bom = { module = "org.junit:junit-bom", version.ref = "junit" }
junit-jupiter = { module = "org.junit.jupiter:junit-jupiter" }
junit-launcher = { module = "org.junit.platform:junit-platform-launcher" }
assertj = { module = "org.assertj:assertj-core", version = "3.27.3" }

[bundles]
testing = ["junit-jupiter", "assertj"]

[plugins]
kotlin-jvm = { id = "org.jetbrains.kotlin.jvm", version.ref = "kotlin" }
spring-boot = { id = "org.springframework.boot", version.ref = "springBoot" }
```

USING IT IN BUILD.GRADLE.KTS

- → `implementation(libs.guava)` — a library entry
- → `platform(libs.junit.bom)` — a BOM entry
- → `libs.bundles.testing` — several at once
- → `alias(libs.plugins.kotlin.jvm)` — in the `plugins { }` block
- → `version.ref` — share one `[versions]` value across entries
- → No `version` — expected to come from a platform/BOM

LIBRARY ENTRY FORMS

TOML	MEANING
`"g:a:1.0"`	Shorthand coordinates
`{ module = "g:a", version = "1.0" }`	Explicit form
`{ module = "g:a", version.ref = "x" }`	Version from `[versions]`
`{ group = "g", name = "a" }`	Split group and name
`version = { strictly = "1.0" }`	Rich version (sheet 12)

MORE CATALOG FEATURES

FEATURE	HOW
Extra catalogs	`versionCatalogs { create("tools") { from(files(…)) } }`
Share a catalog	Publish with the `version-catalog` plugin
Import a published one	`from("com.example:catalog:1.0")`
In build-logic	`from(files("../gradle/libs.versions.toml"))`

⚠ COMMON MISTAKES

- Using `libs.bundles` or `libs` libraries inside `plugins { }` — removed in Gradle 9; only `libs.plugins.*` works there
- Naming a key `junit.jupiter` and a key `junit` — a key cannot be both a leaf and a group
- Expecting a catalog entry to be a dependency — nothing is added until you reference it

✓ TIPS & BEST PRACTICES

- Keep shared versions in `[versions]` and use `version.ref` everywhere else
- Sort entries alphabetically; diff reviews get much easier
- Pair the catalog with a BOM for big families (Spring, Jackson, JUnit)

QUICK SUMMARY

- `libs.versions.toml` — all versions in one file
- `libs.x` — type-safe accessor for `[libraries]`
- `alias(libs.plugins.x)` — plugins from the catalog
- `version.ref` — reuse a named version

Conflicts, Constraints & Platforms

highest-version wins · constraints { } · platform() / BOMs · enforcedPlatform · exclude · strictly / require / prefer · dependency locking

RESOLVING VERSIONS

When two paths in the graph ask for different versions of one module, Gradle picks the **highest** version by default (Maven picks the nearest). **Constraints** and **platforms** (BOMs) let you steer that choice without adding dependencies, and **rich versions** say how firm a version is. Use `dependencyInsight` to see which version won and why.

NOTES

- A BOM is a POM of versions; `platform()` imports it as constraints
- `enforcedPlatform` overrides even higher transitive versions
- Constraints apply only if something else pulls the module in
- Locking (`--write-locks`) freezes dynamic versions for repeatable builds

STEERING RESOLUTION

```
dependencies {
    implementation(platform(libs.spring.boot.bom))       // import a BOM
    implementation("com.fasterxml.jackson.core:jackson-databind")  // no version
    constraints {
        implementation("org.yaml:snakeyaml:2.4") { because("CVE fix") }
    }
    implementation("com.example:client:1.2") {
        exclude(group = "commons-logging", module = "commons-logging")
    }
    implementation("org.slf4j:slf4j-api") {
        version { strictly("[2.0, 3.0[") }               // any 2.x, never 3
    }
}
configurations.all { resolutionStrategy.failOnVersionConflict() }
dependencyLocking { lockAllConfigurations() }
```

CODE EXPLAINED

- → `platform(...)` — versions from the BOM become constraints
- → `constraints { }` — minimum version if the module appears at all
- → `because("…")` — shows up in `dependencyInsight` output
- → `exclude(...)` — drop one transitive dependency
- → `strictly("[2.0, 3.0[")` — a range nothing can override
- → `failOnVersionConflict()` — make every conflict a build error

IN GROOVY DSL

- → `implementation(platform(x))` → `implementation platform(x)`
- → `exclude(group = "a", module = "b")` → `exclude group: 'a', module: 'b'`
- → `version { strictly("…") }` → `version { strictly '…' }`

JAVA VS KOTLIN PROJECTS

- → Same conflict rules for Java and Kotlin dependencies
- → Kotlin — align Kotlin modules with `platform("org.jetbrains.kotlin:kotlin-bom:…")`
- → Kotlin — coroutines: `platform("org.jetbrains.kotlinx:kotlinx-coroutines-bom:…")`

RICH VERSIONS

KEYWORD	MEANING
`require("1.2")`	At least 1.2 (same as a plain version)
`prefer("1.4")`	Use 1.4 if nothing asks for more
`strictly("1.3")`	Exactly 1.3 — fails if impossible
`reject("1.3.1")`	Never pick this version
`"1.+" / "[1.0,2.0)"`	Dynamic / range — lock these

COMMANDS

COMMAND	SHOWS
`dependencyInsight --dependency jackson`	Why this version won
`dependencies --configuration runtimeClasspath`	Resolved tree
`dependencies --write-locks`	Write `gradle.lockfile`
`build --refresh-dependencies`	Re-check remote versions

⚠ COMMON MISTAKES

- Using `enforcedPlatform` in a published library — it forces versions on every consumer; keep it to applications
- Using `exclude` to fix a version clash — it removes the module entirely; use a constraint instead
- Leaving `1.+` versions unlocked — builds change silently when a new release appears

✓ TIPS & BEST PRACTICES

- Add `because("…")` to every constraint so the reason survives review
- Spring Boot: use `SpringBootPlugin.BOM_COORDINATES` instead of hard-coding the BOM
- Fix CVEs in transitive libs with a constraint, not a direct dependency

QUICK SUMMARY

- Highest version wins — Gradle's default
- `platform(bom)` — import aligned versions
- `constraints { }` — set a version without a dependency
- `dependencyInsight` — explain a resolution

Java Plugin & Source Sets

BUILDING JAVA CODE

The `java` plugin adds the compile → process resources → test → jar pipeline and the **source set** model. A source set is a group of source files with its own classpath and tasks: `main` and `test` exist by default, and you can add more. Use `java-library` instead of `java` for anything other projects depend on; it adds the `api` configuration.

NOTES

- Each source set gets tasks like `compile<Name>Java` and configurations like `<name>Implementation`
- Prefer a toolchain (sheet 14) over `sourceCompatibility`
- For extra test types, prefer `jvm-test-suite` (sheet 16)

CONFIGURING JAVA COMPILATION

```
plugins { `java-library` }

java { withSourcesJar(); withJavadocJar() }   // extra JARs to publish
sourceSets {
    main { java.srcDir("src/generated/java") }   // extra source dir
    create("tools") {                            // a new source set
        compileClasspath += sourceSets.main.get().output
        runtimeClasspath += sourceSets.main.get().output
    }
}
tasks.withType<JavaCompile>().configureEach {
    options.encoding = "UTF-8"
    options.compilerArgs.addAll(listOf("-Xlint:all", "-Werror", "-parameters"))
}
```

CODE EXPLAINED

- → `withSourcesJar()` — publish a `-sources.jar` with the library
- → `java.srcDir(...)` — add a folder, keep the default one
- → `create("tools")` — adds `src/tools/java` and its tasks
- → `+= main.output` — let the new set see main's classes
- → `-Xlint:all -Werror` — every warning fails the build
- → `-parameters` — keep parameter names (Spring, Jackson)

IN GROOVY DSL

- → `` `java-library` `` → `id 'java-library'`
- → `create("tools") { }` → `tools { }`; drop `.get()`: `sourceSets.main.output`
- → `listOf("-Xlint:all", …)` → `['-Xlint:all', …]`

JAVA VS KOTLIN PROJECTS

- → Kotlin — `kotlin.srcDir(…)`; default `src/main/kotlin`, `tools` gets `src/tools/kotlin`
- → Kotlin — `compilerOptions { allWarningsAsErrors = true; javaParameters = true }`
- → Kotlin — javadoc ignores Kotlin code; use Dokka for the docs JAR

JAVA PLUGIN TASKS

TASK	DOES
`compileJava`	Compiles `src/main/java`
`processResources`	Copies `src/main/resources`
`classes`	Both of the above
`compileTestJava`	Compiles tests
`test`	Runs tests
`jar`	Packages `main` output

DEFAULT LAYOUT

PATH	HOLDS
`src/main/java`	Production code
`src/main/resources`	Config, templates
`src/test/java`	Test code
`src/test/resources`	Test data
`build/classes/java/main`	Compiled classes
`build/libs`	JARs

⚠ COMMON MISTAKES

- Replacing a source dir with `setSrcDirs` when you meant to add one — the default `src/main/java` disappears
- Setting `sourceCompatibility` but compiling with a newer JDK — you can still call APIs that don't exist on the target
- Using `java` for a shared library — consumers then see every `implementation` dependency as missing at compile time

✓ TIPS & BEST PRACTICES

- Use `options.release` (or a toolchain) to truly target an older Java
- Generated sources: register the generator task and pass it to `srcDir(task)`
- Keep custom source sets rare; most needs fit `main`, `test` or a test suite

QUICK SUMMARY

- `java-library` — Java plugin + `api`
- `sourceSets` — groups of sources with own tasks
- `withSourcesJar()` — publish sources
- `JavaCompile` — configure `javac`

Java Toolchains

TOOLCHAINS

A **toolchain** tells Gradle which JDK to compile, test and run with, independent of the JDK that runs Gradle itself. Gradle finds a matching JDK on the machine or, with the **foojay resolver** plugin, downloads one. This keeps builds reproducible: every developer and CI agent compiles with Java 21 even if their default `java` is 17 or 25.

NOTES

- Gradle 9 needs JVM 17+ to run; toolchains can target Java 8
- Toolchain JDKs are cached in `~/.gradle/jdks`
- Gradle 9.8 supports Java 27 for toolchains and the daemon
- The **daemon JVM** can also be pinned (see commands)

PINNING A JDK

```
// settings.gradle.kts — lets Gradle download missing JDKs
plugins {
    id("org.gradle.toolchains.foojay-resolver-convention") version "1.0.0"
}
// build.gradle.kts
java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
        vendor = JvmVendorSpec.ADOPTIUM          // optional
    }
}
tasks.register<JavaExec>("runOnJava25") {        // one task on another JDK
    javaLauncher = javaToolchains.launcherFor {
        languageVersion = JavaLanguageVersion.of(25)
    }
    mainClass = "com.example.App"
    classpath = sourceSets.main.get().runtimeClasspath
}
```

CODE EXPLAINED

- → `foojay-resolver-convention` — auto-downloads JDKs from foojay.io
- → `languageVersion` — the Java version all JVM tasks use
- → `vendor` — limit to one distribution (Adoptium, Azul…)
- → `javaToolchains.launcherFor` — a JVM for one specific task
- → `compilerFor` / `javadocToolFor` — the same for other tools

IN GROOVY DSL

- → `settings.gradle`: `id '…foojay-resolver-convention' version '1.0.0'`
- → `register<JavaExec>("x")` → `register('x', JavaExec)`
- → `sourceSets.main.get().runtimeClasspath` → `sourceSets.main.runtimeClasspath`

JAVA VS KOTLIN PROJECTS

- → Java — `java { toolchain { languageVersion = … } }`
- → Kotlin — `kotlin { jvmToolchain(21) }` sets it for Kotlin and Java tasks
- → Kotlin — `jvmTarget` follows the toolchain unless you set it

COMMANDS

COMMAND	DOES
`./gradlew javaToolchains`	List JDKs Gradle detected
`./gradlew updateDaemonJvm --jvm-version=21`	Pin the JVM that runs Gradle

GRADLE.PROPERTIES SWITCHES

PROPERTY	EFFECT
`org.gradle.java.installations.auto-download=false`	Never download JDKs
`org.gradle.java.installations.auto-detect=false`	Ignore JDKs found on the system
`org.gradle.java.installations.paths=/opt/jdk-21`	Extra JDK locations

⚠ COMMON MISTAKES

- Setting `sourceCompatibility = 21` and also a toolchain — redundant; the toolchain already sets the release
- Forgetting the foojay plugin on fresh CI agents — the build fails with "no matching toolchain"
- Using `updateDaemonJvm` but not committing `gradle/gradle-daemon-jvm.properties`

✓ TIPS & BEST PRACTICES

- Pin the toolchain in a convention plugin so all subprojects match
- Test on several JDKs by registering extra `Test` tasks with `javaLauncher`
- In CI, cache `~/.gradle/jdks` alongside the dependency cache

QUICK SUMMARY

- `java { toolchain { } }` — choose the build JDK
- foojay resolver — auto-download JDKs
- `launcherFor { }` — per-task JVM
- `updateDaemonJvm` — pin Gradle's own JVM

Application Plugin & JARs

SHIPPING AN APP

The `application` plugin adds a `run` task, start scripts and ZIP/TAR distributions that bundle your JAR with its dependencies in `lib/`. A plain `jar` contains only your classes, so `java -jar` fails unless every dependency is on the classpath. For one self-contained file, use the **Shadow** plugin to build a fat JAR.

RUNNABLE APP + FAT JAR

```
plugins {
    application
    id("com.gradleup.shadow") version "9.0.0"
}
application {
    mainClass = "com.example.App"
    applicationDefaultJvmArgs = listOf("-Xmx512m")
}
tasks.jar {
    manifest {
        attributes("Main-Class" to "com.example.App",
                   "Implementation-Version" to project.version)
    }
}
tasks.shadowJar {
    archiveClassifier = "all"          // build/libs/app-1.0-all.jar
    mergeServiceFiles()                // merge META-INF/services
}
```

NOTES

- Archives are reproducible by default in Gradle 9 (fixed order and timestamps)
- JAR name: `<project>-<version>.jar` in `build/libs/`
- `installDist` output: `build/install/<app>/bin/<app>`

CODE EXPLAINED

- → `mainClass` — the class with `main()` used by `run` and scripts
- → `applicationDefaultJvmArgs` — baked into the start scripts
- → `manifest { attributes }` — entries in `META-INF/MANIFEST.MF`
- → `shadowJar` — classes + all runtime dependencies in one JAR
- → `mergeServiceFiles()` — keeps ServiceLoader files from clashing

IN GROOVY DSL

- → `listOf("-Xmx512m")` → `['-Xmx512m']`
- → `attributes("Main-Class" to "…")` → `attributes('Main-Class': '…')`
- → `tasks.jar { }` → `tasks.named('jar') { }`

JAVA VS KOTLIN PROJECTS

- → Java — `mainClass = "com.example.App"`, the class with `main()`
- → Kotlin — top-level `fun main()` in App.kt → `mainClass = "com.example.AppKt"`
- → Kotlin — `shadowJar` also bundles kotlin-stdlib

TASKS ADDED

TASK	OUTPUT
`run`	Runs `mainClass` from the build
`jar`	`build/libs/app-1.0.jar`
`startScripts`	Unix + Windows launchers
`installDist`	`build/install/app/`
`distZip / distTar`	`build/distributions/app-1.0.zip`
`shadowJar`	`build/libs/app-1.0-all.jar`

RUNNING IT

COMMAND	NOTE
`./gradlew run --args="--port 8080"`	Pass program arguments
`./gradlew installDist`	Then `build/install/app/bin/app`
`java -jar build/libs/app-1.0-all.jar`	Fat JAR, no classpath
`./gradlew run --debug-jvm`	Wait for a debugger on 5005

⚠ COMMON MISTAKES

- Running `java -jar` on the plain JAR — `NoClassDefFoundError`, because dependencies aren't inside it
- Reading `System.in` under `./gradlew run` — set `standardInput = System.in` on the `run` task
- Using the old `com.github.johnrengelman.shadow` id — use `com.gradleup.shadow`

✓ TIPS & BEST PRACTICES

- Prefer `installDist` + `lib/` folder over a fat JAR when libraries need signing
- Use `relocate("com.google", "shadow.google")` in a shaded library to avoid clashes
- Set `version` in `gradle.properties` so CI can override it with `-Pversion=`

QUICK SUMMARY

- `application { mainClass }` — make it runnable
- `./gradlew run` — run from the build
- `installDist` / `distZip` — ship with start scripts
- `shadowJar` — one fat JAR

Testing

RUNNING TESTS

The `test` task runs the tests in `src/test` and writes HTML reports to `build/reports/tests/test/`. Call `useJUnitPlatform()` for JUnit 5/6, TestNG via its engine, Spock or Kotest. For extra kinds of tests, such as integration tests, the **JVM Test Suite** plugin creates a source set, configurations and a `Test` task in one block.

GRADLE 9 CHANGES

- Declare `junit-platform-launcher` as `testRuntimeOnly` yourself
- `test` fails if it finds **no** tests (`failOnNoDiscoveredTests`)
- Custom `Test` tasks no longer inherit the `test` classpath
- Tests are cached: unchanged code → `UP-TO-DATE`

TEST CONFIGURATION

```
import org.gradle.api.tasks.testing.logging.TestExceptionFormat.FULL

dependencies {
    testImplementation(platform("org.junit:junit-bom:5.13.4"))
    testImplementation("org.junit.jupiter:junit-jupiter")
    testRuntimeOnly("org.junit.platform:junit-platform-launcher")
}
tasks.test {
    useJUnitPlatform { excludeTags("slow") }
    maxParallelForks = 4
    testLogging { events("failed", "skipped"); exceptionFormat = FULL }
}
testing.suites {                                  // jvm-test-suite plugin
    register<JvmTestSuite>("integrationTest") {
        useJUnitJupiter()
        dependencies { implementation(project()) }
    }
}
```

CODE EXPLAINED

- → `junit-bom` — one version for all JUnit modules
- → `excludeTags("slow")` — skip `@Tag("slow")` tests
- → `maxParallelForks` — run test classes in parallel JVMs
- → `exceptionFormat = FULL` — full stack traces in the console
- → `register<JvmTestSuite>` — adds `src/integrationTest` + task
- → `implementation(project())` — the suite sees main's code

IN GROOVY DSL

- → `exceptionFormat = FULL` → `exceptionFormat = 'full'` , no import
- → `tasks.test { }` → `tasks.named('test') { }` ; `excludeTags 'slow'`
- → `register<JvmTestSuite>("x")` → `x(JvmTestSuite) { }`

JAVA VS KOTLIN PROJECTS

- → Same `test` task and JUnit Platform for Java and Kotlin
- → Kotlin — `testImplementation(kotlin("test"))` picks kotlin-test-junit5
- → Kotlin — backtick test names: `fun `adds two numbers`()`

SELECTING TESTS

COMMAND	RUNS
`test --tests 'com.example.UserTest'`	One class
`test --tests '*Repository*'`	Pattern match
`test --tests 'UserTest.saves*'`	Matching methods
`test --fail-fast`	Stop at first failure
`integrationTest`	The custom suite
`test --rerun`	Run even if up to date

TEST TASK SETTINGS

SETTING	DOES
`maxHeapSize = "1g"`	Test JVM heap
`jvmArgs("…")`	Extra JVM flags
`systemProperty("k", "v")`	Pass a system property
`environment("K", "v")`	Set an env variable
`forkEvery = 100`	New JVM every N classes
`failOnNoDiscoveredTests = false`	Allow empty test runs

⚠ COMMON MISTAKES

- Calling `useJUnitPlatform()` but adding JUnit 4 tests only — no engine finds them; add `junit-vintage-engine`
- Registering a plain `Test` task for integration tests — in Gradle 9 it has no classpath; use a test suite
- Using `--rerun-tasks` to re-run tests — it reruns everything; `test --rerun` is enough

✓ TIPS & BEST PRACTICES

- Open `build/reports/tests/test/index.html` for full results and output
- Make `check` depend on `integrationTest` so `build` runs the suite too
- Use `--scan` to see slowest tests and flaky retries

QUICK SUMMARY

- `useJUnitPlatform()` — run JUnit Platform engines
- `--tests` — filter by class or method
- `testing.suites` — extra test types
- `testLogging` — console output

Multi-Project Builds

include(":app", ":lib") · rootProject.name · project(":lib") dependency · subproject layout · :app:build · avoid allprojects

MANY PROJECTS, ONE BUILD

A **multi-project build** has one `settings.gradle.kts` that `includes` several subprojects, each with its own build script. Projects depend on each other with `project(":core")`, and Gradle builds them in the right order, in parallel when allowed. Share configuration with **convention plugins** (sheet 18) rather than `allprojects { }` / `subprojects { }` blocks.

NOTES

- `include("lib:payments")` maps to folder `lib/payments`
- The root project often has no code, only settings
- `TYPESAFE_PROJECT_ACCESSORS` enables `projects.core`
- `org.gradle.parallel=true` builds independent projects at once

SETTINGS + TWO SUBPROJECTS

```
// settings.gradle.kts
rootProject.name = "shop"
include("app", "core", "lib:payments")       // lib/payments → :lib:payments
enableFeaturePreview("TYPESAFE_PROJECT_ACCESSORS")

// core/build.gradle.kts
plugins { `java-library` }
dependencies { api(libs.jackson.databind) }

// app/build.gradle.kts
plugins { application }
dependencies {
    implementation(project(":core"))
    implementation(projects.lib.payments)     // type-safe accessor
}
application { mainClass = "shop.App" }
```

CODE EXPLAINED

- → `rootProject.name` — the build's name; set it explicitly
- → `include(…)` — each path becomes a subproject
- → `api(...)` in `core` — `app` also sees Jackson
- → `project(":core")` — string path to a sibling project
- → `projects.lib.payments` — the same, checked at compile time

IN GROOVY DSL

- → `settings.gradle`: `include 'app', 'core', 'lib:payments'`
- → `api(libs.jackson.databind)` → `api libs.jackson.databind`
- → `projects.lib.payments` — works in Groovy too

JAVA VS KOTLIN PROJECTS

- → Java — each subproject applies `java-library` or `application`
- → Kotlin — declare `kotlin("jvm") version "…" apply false` once in the root
- → Kotlin — subprojects then apply `kotlin("jvm")` with no version

COMMANDS

COMMAND	RUNS
`./gradlew build`	`build` in every project
`./gradlew :app:build`	`app` and what it depends on
`./gradlew :core:test`	Only core's tests
`./gradlew projects`	Print the project tree
`./gradlew :app:dependencies`	One project's dependencies
`./gradlew build --parallel`	Projects in parallel

SHARING CONFIG

APPROACH	VERDICT
`Convention plugin`	✔ Preferred — opt-in per project
`Version catalog`	✔ For versions only
`gradle.properties`	✔ For flags shared by all
`subprojects { }`	✖ Couples projects, breaks isolation
`allprojects { }`	✖ Same, plus applies to root

⚠ COMMON MISTAKES

- Reaching into another project (`project(":core").tasks…`) — couples configuration and blocks Isolated Projects
- Creating a folder but not `include` -ing it — Gradle ignores it without warning
- Putting code in the root project of a multi-project build — tasks then run twice in confusing ways

✓ TIPS & BEST PRACTICES

- Group libraries under `libs/` or `modules/` and keep `app` thin
- Run the narrowest task path (`:app:test`) for fast feedback
- Use `--configure-on-demand` or Isolated Projects (incubating in 9.7+) for huge builds

QUICK SUMMARY

- `include("a", "b")` — add subprojects
- `project(":a")` — depend on a sibling
- `:a:task` — run in one project
- Convention plugins — share config safely

Convention Plugins (build-logic)

SHARING BUILD LOGIC

A **convention plugin** is a build script turned into a plugin: shared setup (toolchain, test config, linting) lives in one file, and each project opts in with `plugins { id("java-conventions") }`. Put them in a `build-logic` **included build** (or the older `buildSrc`). The file name of a precompiled script plugin becomes its id.

NOTES

- `kotlin-dsl` plugin compiles `src/main/kotlin/*.gradle.kts` into plugins
- Changing `buildSrc` invalidates the whole build; `build-logic` less so
- `libs` accessors are not generated inside precompiled plugins
- Read the catalog there with `the<VersionCatalogsExtension>()`

A BUILD-LOGIC SETUP

```
// settings.gradle.kts (root)
pluginManagement { includeBuild("build-logic") }

// build-logic/build.gradle.kts
plugins { `kotlin-dsl` }
repositories { gradlePluginPortal() }

// build-logic/src/main/kotlin/java-conventions.gradle.kts
plugins { `java-library` }
val libs = the<VersionCatalogsExtension>().named("libs")
java { toolchain { languageVersion = JavaLanguageVersion.of(21) } }
dependencies {
    testImplementation(libs.findLibrary("junit-jupiter").get())
}
tasks.withType<Test>().configureEach { useJUnitPlatform() }

// core/build.gradle.kts
plugins { id("java-conventions") }
```

CODE EXPLAINED

- → `includeBuild("build-logic")` — plugins from this build are usable
- → `kotlin-dsl` — turns `.gradle.kts` files into plugins
- → `java-conventions.gradle.kts` — id is `java-conventions`
- → `the<VersionCatalogsExtension>()` — untyped catalog lookup
- → `plugins { id(…) }` — a project opts in, no version needed

IN GROOVY DSL

- → `` `kotlin-dsl` `` → `id 'groovy-gradle-plugin'`
- → Script: `src/main/groovy/java-conventions.gradle`
- → Catalog: `versionCatalogs.named('libs').findLibrary('junit-jupiter').get()`

JAVA VS KOTLIN PROJECTS

- → `kotlin-dsl` is for the build logic — the projects can be Java or Kotlin
- → Kotlin conventions — add `org.jetbrains.kotlin:kotlin-gradle-plugin` to build-logic `dependencies`
- → …then `plugins { kotlin("jvm") }` with no version in the script

BUILDSRC VS BUILD-LOGIC

	BUILDSRC	BUILD-LOGIC
Setup	Folder name is magic	`includeBuild` in settings
Rebuild cost	Any change rebuilds all	Only affected plugins
Several logic builds	No	Yes
Recommended	Small builds	Most builds

FILE → PLUGIN ID

FILE	ID
`java-conventions.gradle.kts`	java-conventions
`myorg.library.gradle.kts`	myorg.library
`com/acme/app.gradle.kts + package`	com.acme.app

⚠ COMMON MISTAKES

- Using `libs.junit.jupiter` inside a precompiled plugin — the accessor doesn't exist there; use `findLibrary`
- Giving the plugin a version in `plugins { }` — local convention plugins are versionless
- Forgetting `repositories { }` in `build-logic` — external plugins it applies can't be resolved

✓ TIPS & BEST PRACTICES

- Make one plugin per role: `java-conventions`, `library-conventions`, `app-conventions`
- Share the root catalog with `build-logic` from its own settings file (sheet 11)
- Layer plugins: `library-conventions` applies `java-conventions`

QUICK SUMMARY

- `build-logic` — an included build for plugins
- `kotlin-dsl` — precompiled script plugins
- File name — becomes the plugin id
- `plugins { id("x-conventions") }` — opt in

Composite Builds

COMBINING SEPARATE BUILDS

A **composite build** joins complete, independent builds, each with its own settings file, so they act as one. The common case: you are fixing a library and an app that uses it. `includeBuild("../json-lib")` makes Gradle build the library from source and **substitute** it for the published `com.example:json-lib` dependency, with no version bump or `publishToMavenLocal`. Included builds are also how `build-logic` (sheet 18) delivers plugins.

NOTES

- Substitution is automatic when `group:name` match the included project
- Included builds run on the including build's Gradle version
- Run a task in an included build: `./gradlew :json-lib:test`
- Remove the `includeBuild` line to go back to the published JAR

DEVELOPING AN APP AND A LIBRARY TOGETHER

```
// app/settings.gradle.kts
pluginManagement {
    includeBuild("../my-plugins")             // plugins built from source
}
rootProject.name = "app"
includeBuild("../json-lib")                   // swap in a local checkout

// Explicit form, when coordinates don't match automatically:
// includeBuild("../json-lib") {
//     dependencySubstitution {
//         substitute(module("com.example:json-lib")).using(project(":"))
//     }
// }

// app/build.gradle.kts — unchanged
dependencies { implementation("com.example:json-lib:2.3.0") }
```

CODE EXPLAINED

- → `pluginManagement { includeBuild }` — plugins from a local build
- → `includeBuild("../json-lib")` — library built from source
- → `substitute(…).using(project(":"))` — manual mapping
- → The app's dependency line does not change
- → One-off without editing settings: `--include-build ../json-lib`

IN GROOVY DSL

- → `settings.gradle` — same calls: `includeBuild '../json-lib'`
- → `substitute(module('…')).using(project(':'))` — same, single quotes
- → `implementation("g:a:v")` → `implementation 'g:a:v'`

JAVA VS KOTLIN PROJECTS

- → Language doesn't matter — any JVM build can be included
- → A Kotlin app can include a Java library, and the reverse
- → Substitution swaps the JAR for the included build's `jar` output

COMPOSITE VS MULTI-PROJECT

	MULTI-PROJECT	COMPOSITE
Settings files	One	One per included build
Links	`project(":lib")`	Normal `g:a:v` coordinates
Shared config	Convention plugins	Nothing shared
Typical use	One repo, many modules	Separate repos worked on together

COMMANDS

COMMAND	DOES
`./gradlew build --include-build ../lib`	Composite for one run
`./gradlew :json-lib:test`	Task in the included build
`./gradlew dependencyInsight --dependency json-lib`	Shows `project :json-lib`

⚠ COMMON MISTAKES

- Including a build whose `group` differs from the coordinates you use — nothing is substituted; add `dependencySubstitution`
- Committing a developer-only `includeBuild` — CI then needs the sibling checkout; use `--include-build` instead
- Expecting `allprojects { }` to reach included builds — they are separate builds

✓ TIPS & BEST PRACTICES

- Guard optional includes: `if (file("../json-lib").exists()) includeBuild("../json-lib")`
- IntelliJ imports included builds, so you can debug into library source
- Use composites instead of `mavenLocal()` for cross-repo work

QUICK SUMMARY

- `includeBuild(path)` — join another build
- Automatic substitution — by `group:name`
- `--include-build` — composite for one run
- `pluginManagement { includeBuild }` — plugins from source

Build Cache & Configuration Cache

TWO DIFFERENT CACHES

The **build cache** stores task **outputs** keyed by their inputs, so a task whose inputs were seen before (on any branch, or on another machine with a remote cache) is restored `FROM-CACHE` instead of run. The **configuration cache** stores the **task graph** after configuration, so the next run with the same tasks skips running build scripts entirely. Gradle 9 treats the configuration cache as the preferred mode; turn both on in `gradle.properties`.

TURNING THEM ON + A REMOTE CACHE

```
// gradle.properties
//   org.gradle.caching=true
//   org.gradle.configuration-cache=true
//   org.gradle.configuration-cache.problems=warn   # while migrating

// settings.gradle.kts — share outputs between CI and developers
buildCache {
    local { isEnabled = true }
    remote<HttpBuildCache> {
        url = uri("https://cache.example.com/cache/")
        isPush = providers.environmentVariable("CI").isPresent  // CI writes
        credentials { username = "ci"; password = providers
            .environmentVariable("CACHE_PASSWORD").orNull }
    }
}
```

NOTES

- `UP-TO-DATE` — outputs already in `build/` ; `FROM-CACHE` — restored
- Cached tasks need declared inputs/outputs (sheet 22)
- Config-cache problems are listed in an HTML report under `build/reports/configuration-cache/`
- Local cache: `~/.gradle/caches/build-cache-1`

CODE EXPLAINED

- → `org.gradle.caching=true` — same as `--build-cache` every run
- → `configuration-cache=true` — same as `--configuration-cache`
- → `problems=warn` — report problems without failing
- → `remote<HttpBuildCache>` — a shared HTTP cache node
- → `isPush` only on CI — developers read, CI writes

IN GROOVY DSL

- → `local { isEnabled = true }` → `local { enabled = true }`
- → `remote<HttpBuildCache> { }` → `remote(HttpBuildCache) { }`
- → `isPush = ….isPresent` → `push = ….present`

JAVA VS KOTLIN PROJECTS

- → `compileJava` and `compileKotlin` are both cacheable
- → Kotlin — compiles in its own Kotlin daemon; tune with `kotlin.daemon.jvmargs`
- → Kotlin plugin supports the configuration cache — keep it current

WHAT YOU'LL SEE

MESSAGE	MEANING
`Configuration cache entry stored.`	First run with these tasks
`Reusing configuration cache.`	Scripts skipped
`:compileJava FROM-CACHE`	Output restored
`:test UP-TO-DATE`	Nothing changed
`Calculating task graph as … has changed`	A build file changed

COMMON CONFIG-CACHE PROBLEMS

PROBLEM	FIX
`project` used at execution time	Capture values into `val`s before `doLast`
`System.getenv()` read	`providers.environmentVariable()`
`project.exec { }`	`providers.exec` or inject `ExecOperations`
`buildDir` / `project.file` in actions	`layout.buildDirectory`, captured early

⚠ COMMON MISTAKES

- Confusing the two caches — the build cache saves task work; the configuration cache saves script evaluation
- Letting every developer push to the remote cache — one bad machine can poison it
- Using absolute paths as task inputs — cache keys differ per machine; use `@PathSensitive(RELATIVE)`

✓ TIPS & BEST PRACTICES

- Try `./gradlew build --configuration-cache` before enabling it in properties
- Run `./gradlew clean build` twice: the second should be mostly `FROM-CACHE`
- A build scan's Performance tab shows cache hits and misses

QUICK SUMMARY

- Build cache — reuse task outputs
- Configuration cache — reuse the task graph
- `org.gradle.caching=true` — enable the build cache
- `org.gradle.configuration-cache=true` — enable the config cache

Daemon & Build Tuning

org.gradle.daemon · org.gradle.jvmargs · org.gradle.parallel · --stop / --status · offline mode · build scans (--scan) · --profile

MAKING BUILDS FAST

The **daemon** is a background JVM that keeps classes, caches and the file-system state warm between builds; it is on by default. Most speed comes from a few switches in `gradle.properties`: enough heap, parallel project execution, and the two caches. Measure before tuning: a **build scan** (`--scan`) or `--profile` shows where time goes.

NOTES

- A new daemon starts if JVM args or Java version differ
- Daemons stop after 3 hours idle by default
- `~/.gradle/gradle.properties` overrides the project file per machine
- File-system watching is on by default on all major OSes

GRADLE.PROPERTIES FOR A FAST BUILD

```
# Commit the settings the whole team should share
org.gradle.jvmargs=-Xmx4g -XX:MaxMetaspaceSize=1g -Dfile.encoding=UTF-8
org.gradle.parallel=true
org.gradle.caching=true
org.gradle.configuration-cache=true
org.gradle.workers.max=8
org.gradle.daemon.idletimeout=3600000
org.gradle.vfs.watch=true
kotlin.daemon.jvmargs=-Xmx2g
```

CODE EXPLAINED

- → `jvmargs` — heap for the daemon; the most common fix for slow builds
- → `parallel` — build independent projects at the same time
- → `caching` / `configuration-cache` — sheet 20
- → `workers.max` — cap parallel work (default: CPU cores)
- → `idletimeout` — ms before an idle daemon exits
- → `kotlin.daemon.jvmargs` — heap for the Kotlin compiler

DAEMON COMMANDS

COMMAND	DOES
`./gradlew --status`	List running daemons
`./gradlew --stop`	Stop all daemons of this version
`./gradlew build --no-daemon`	One-off JVM
`./gradlew build --offline`	No network

MEASURING

TOOL	GIVES
`--scan`	Web report: timeline, cache, tests
`--profile`	HTML in `build/reports/profile`
`--info`	Why tasks ran
`--dry-run`	What would run

SPEED CHECKLIST

CHECK	WHY
`Avoid clean`	Keeps incremental work
`Lazy task APIs`	Less configuration
`Narrow task paths`	`:app:test`, not `test`
`Up-to-date plugins`	Older ones are slower

⚠ COMMON MISTAKES

- Setting `org.gradle.jvmargs` without `-Xmx` — the default heap is small and builds thrash in GC
- Using `--no-daemon` locally — every build pays JVM startup and cold caches
- Killing daemons with `kill -9` — use `./gradlew --stop` so locks are released

✓ TIPS & BEST PRACTICES

- Put machine-specific heap sizes in `~/.gradle/gradle.properties`
- Exclude the project and `~/.gradle` from antivirus scans on Windows
- Compare two scans side by side to find what slowed a build

QUICK SUMMARY

- Daemon — warm JVM reused between builds
- `org.gradle.jvmargs` — daemon heap
- `org.gradle.parallel` — parallel projects
- `--scan` / `--profile` — measure first

Writing Custom Tasks

TASK CLASSES

For anything beyond a few lines, write a **task class**: an `abstract` class extending `DefaultTask` with annotated lazy properties and one `@TaskAction` method. The annotations tell Gradle what the inputs and outputs are, which makes the task **incremental** (`UP-TO-DATE`) and, with `@CacheableTask`, **cacheable**. Keep the class in `build-logic` so it can be tested and reused.

A CACHEABLE TASK

```
@CacheableTask
abstract class GenerateVersionFile : DefaultTask() {
    @get:Input abstract val version: Property<String>
    @get:InputFile @get:PathSensitive(PathSensitivity.RELATIVE)
    abstract val template: RegularFileProperty
    @get:OutputFile abstract val outputFile: RegularFileProperty

    @TaskAction fun generate() {
        val text = template.get().asFile.readText()
        val out = text.replace("@VERSION@", version.get())
        outputFile.get().asFile.writeText(out)
    }
}
tasks.register<GenerateVersionFile>("generateVersionFile") {
    version = project.version.toString()
    template = layout.projectDirectory.file("src/version.tmpl")
    outputFile = layout.buildDirectory.file("generated/version.txt")
}
```

NOTES

- Gradle generates the property implementations for `abstract` getters
- In Kotlin, annotate with `@get:Input` (the getter), not `@Input`
- Inject services (`ExecOperations` , `FileSystemOperations`) instead of using `project`

CODE EXPLAINED

- → `@CacheableTask` — outputs may be stored in the build cache
- → `abstract val … : Property<String>` — Gradle implements it
- → `@PathSensitive(RELATIVE)` — cache key ignores absolute paths
- → `@OutputFile` — Gradle creates parent folders and tracks the file
- → `@TaskAction` — the method that does the work
- → `register<T>` — wire values in at configuration time

IN GROOVY DSL

- → `abstract class X : DefaultTask()` → `abstract class X extends DefaultTask`
- → `@get:Input abstract val v: Property<String>` → `@Input abstract Property<String> getV()`
- → `register<X>("x")` → `register('x', X)`

JAVA VS KOTLIN PROJECTS

- → Java — `public abstract class X extends DefaultTask` , abstract getters
- → Kotlin — `@get:` puts the annotation on the getter Gradle reads
- → In build-logic: `src/main/java` or `src/main/kotlin`

INPUT / OUTPUT ANNOTATIONS

ANNOTATION	FOR
`@Input`	Strings, numbers, enums
`@InputFile / @InputFiles`	One / many files
`@InputDirectory`	A directory tree
`@Classpath`	Classpath (order matters)
`@OutputFile / @OutputDirectory`	What the task writes
`@Internal`	Not an input (e.g. a logger)

INJECTABLE SERVICES

SERVICE	REPLACES
`ExecOperations`	`project.exec` / `javaexec`
`FileSystemOperations`	`project.copy` / `delete`
`ArchiveOperations`	`project.zipTree`
`WorkerExecutor`	Parallel work inside a task
`ProviderFactory`	`project.providers`

⚠ COMMON MISTAKES

- Writing `@Input` instead of `@get:Input` in Kotlin — it lands on the property, where Gradle never sees it
- Forgetting `abstract` on the class — Gradle 9 requires it when properties are abstract or `@Inject` is used
- Not declaring outputs — the task can never be `UP-TO-DATE` and always runs

✓ TIPS & BEST PRACTICES

- Inject a service: `@get:Inject abstract val exec: ExecOperations`
- Use `@get:Incremental` + `InputChanges` to process only changed files
- Test task classes with Gradle TestKit (`GradleRunner`)

QUICK SUMMARY

- `abstract class X : DefaultTask()` — a task type
- `@get:Input` / `@get:OutputFile` — declare I/O
- `@TaskAction` — the work
- `@CacheableTask` — opt into the build cache

Publishing Libraries

PUBLISHING TO A REPOSITORY

The `maven-publish` plugin turns a component (your JAR plus its dependency metadata) into a **publication** and uploads it to Maven repositories. Gradle also publishes **Gradle Module Metadata** (`.module`) next to the POM, which preserves features like `api` vs `implementation` and platforms. Add `signing` for Maven Central, which also requires sources and Javadoc JARs.

MAVEN-PUBLISH + SIGNING

```
plugins { `java-library`; `maven-publish`; signing }
java { withSourcesJar(); withJavadocJar() }

publishing {
    publications {
        create<MavenPublication>("maven") {
            from(components["java"])
            artifactId = "json-lib"
            pom { name = "JSON Lib"; url = "https://example.com" }
        }
    }
    repositories {
        maven("https://repo.example.com/releases") {
            name = "company"
            credentials(PasswordCredentials::class)
        }
    }
}
signing { sign(publishing.publications["maven"]) }
```

NOTES

- Maven Central uploads now go through the **Central Portal** (OSSRH closed in 2025)
- Community plugins (e.g. `com.vanniktech.maven.publish`, JReleaser) handle Central
- Coordinates: `group`, project name (or `artifactId`), `version`

CODE EXPLAINED

- → `from(components["java"])` — the JAR plus dependency metadata
- → `artifactId` — defaults to the project name
- → `pom { }` — name, license, SCM, developers (required by Central)
- → `name = "company"` — gives the task name and credential keys
- → `sign(...)` — adds `.asc` signatures for every file

IN GROOVY DSL

- → `create<MavenPublication>("maven")` → `maven(MavenPublication) { }`
- → `components["java"]` → `components.java`
- → `sign(publishing.publications["maven"])` → `sign publishing.publications.maven`

JAVA VS KOTLIN PROJECTS

- → Same `components["java"]` for Java and Kotlin/JVM libraries
- → Kotlin — javadoc ignores Kotlin code; use Dokka for the -javadoc.jar
- → Kotlin — `withSourcesJar()` includes the .kt sources

PUBLISHING TASKS

TASK	DOES
`publishToMavenLocal`	Install to `~/.m2/repository`
`publish`	All publications → all repos
`publishMavenPublicationToCompanyRepository`	One pair
`generatePomFileForMavenPublication`	Just write the POM

SIGNING KEYS (GRADLE.PROPERTIES)

PROPERTY	VALUE
`signing.keyId`	Last 8 chars of the key id
`signing.password`	Key passphrase
`signing.secretKeyRingFile`	Path to the key ring
`useInMemoryPgpKeys(key, pw)`	CI: key from env var

⚠ COMMON MISTAKES

- Publishing with `enforcedPlatform` or dynamic versions — consumers inherit forced or unpredictable versions
- Forgetting `withSourcesJar()` / `withJavadocJar()` — Maven Central rejects the upload
- Testing with `publishToMavenLocal` and leaving `mavenLocal()` in consumers — use a composite build instead

✓ TIPS & BEST PRACTICES

- Set `group` and `version` once in `gradle.properties`
- In CI, pass keys as `ORG_GRADLE_PROJECT_signingKey` and use in-memory keys
- Inspect output in `build/publications/maven/` before uploading

QUICK SUMMARY

- `maven-publish` — publish to Maven repos
- `create<MavenPublication>` — define what to publish
- `publishToMavenLocal` — local install
- `signing { sign(…) }` — required for Central

Spring Boot, Kotlin & Android Plugins

FRAMEWORK PLUGINS

Frameworks ship Gradle plugins that add their own tasks and conventions. The **Spring Boot** plugin builds an executable `bootJar` and runs the app with `bootRun`; the **Kotlin** plugin compiles Kotlin and adds compiler plugins like `plugin.spring`. The **Android Gradle Plugin (AGP)** replaces the `java` plugin entirely and must be paired with a supported Gradle version. This sheet covers the build side only; see the Spring set for Spring itself.

VERSION PAIRING (OCT 2026)

- AGP **9.4** needs Gradle **9.6+** and JDK 17
- AGP 9 compiles Kotlin itself (built-in Kotlin) — no `kotlin-android` plugin
- Gradle 9 needs Kotlin Gradle Plugin **2.0+**, AGP **8.4+**
- Spring Boot 4 supports Gradle 9; check its docs for the minimum

SPRING BOOT + KOTLIN

```
import org.springframework.boot.gradle.plugin.SpringBootPlugin

plugins {
    id("org.springframework.boot") version "4.1.0"
    kotlin("jvm") version "2.2.20"
    kotlin("plugin.spring") version "2.2.20"   // opens Spring classes
}
dependencies {
    implementation(platform(SpringBootPlugin.BOM_COORDINATES))
    implementation("org.springframework.boot:spring-boot-starter-web")
    testImplementation("org.springframework.boot:spring-boot-starter-test")
}
kotlin { jvmToolchain(21) }
tasks.bootJar { archiveFileName = "app.jar" }  // build/libs/app.jar
```

CODE EXPLAINED

- → `org.springframework.boot` — adds `bootJar`, `bootRun`
- → `kotlin("plugin.spring")` — makes Spring beans `open`
- → `platform(BOM_COORDINATES)` — Boot's versions without the extra plugin
- → Starters need no version once the BOM is imported
- → `jvmToolchain(21)` — Kotlin shortcut for a Java toolchain

IN GROOVY DSL

- → `kotlin("jvm") version "…"` → `id 'org.jetbrains.kotlin.jvm' version '…'`
- → `kotlin("plugin.spring")` → `id 'org.jetbrains.kotlin.plugin.spring'`
- → `tasks.bootJar { }` → `tasks.named('bootJar') { }`

JAVA VS KOTLIN PROJECTS

- → Java — drop the Kotlin plugins; `java { toolchain { … } }` replaces `jvmToolchain`
- → Kotlin — add `kotlin("plugin.jpa")` for no-arg `@Entity` classes
- → Kotlin — add `jackson-module-kotlin` for data-class JSON

SPRING BOOT TASKS

TASK	DOES
`bootRun`	Run the app
`bootJar`	Executable fat JAR
`bootBuildImage`	OCI image via Buildpacks
`bootTestRun`	Run with test classpath

ANDROID TASKS (AGP)

TASK	DOES
`assembleDebug`	Debug APK
`bundleRelease`	Release App Bundle (.aab)
`installDebug`	Install on a device
`connectedAndroidTest`	Instrumented tests
`lint`	Android Lint

PLUGIN IDS

PLUGIN	KOTLIN DSL
`Kotlin JVM`	`kotlin("jvm")`
`Android app`	`id("com.android.application")`
`Android lib`	`id("com.android.library")`
`Compose`	`kotlin("plugin.compose")`

⚠ COMMON MISTAKES

- Applying both `io.spring.dependency-management` and `platform()` — two mechanisms fighting; pick one (`platform()` is native)
- Upgrading AGP without upgrading Gradle — AGP checks the Gradle version and fails
- Keeping `kotlin-android` with AGP 9 — it clashes with built-in Kotlin

✓ TIPS & BEST PRACTICES

- Boot also builds a `-plain.jar`; turn it off with `tasks.jar { enabled = false }`
- Let Android Studio's AGP Upgrade Assistant bump AGP and Gradle together
- Keep Kotlin plugin versions in the catalog so all `kotlin(…)` plugins match

QUICK SUMMARY

- `bootJar` / `bootRun` — Spring Boot build
- `kotlin("jvm")` — Kotlin compilation
- `com.android.application` — Android (AGP)
- Pair AGP with its minimum Gradle version

CI & Troubleshooting

BUILDS ON CI

On CI, run the wrapper with the same tasks developers use, and cache `~/.gradle` between runs. On GitHub Actions, `gradle/actions/setup-gradle` handles caching, wrapper validation and build-scan links. When a build fails, add logging and a stack trace first, then inspect the dependency graph; clear caches only as a last resort.

CI DEFAULTS

- Gradle detects CI and uses plain console output
- Use `--no-daemon` only if agents are thrown away anyway
- Run `./gradlew build` — not separate `compile`, `test` steps
- Publish test reports from `build/reports/tests/`

GITHUB ACTIONS WORKFLOW

```yaml
# .github/workflows/build.yml
name: build
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: 21 }
      - uses: gradle/actions/setup-gradle@v4   # cache + wrapper check
      - run: ./gradlew build --scan
```

CODE EXPLAINED

- → `setup-java` — JDK that runs Gradle (17+)
- → `setup-gradle` — caches `~/.gradle`, validates the wrapper
- → `--scan` — link to a build scan in the job log
- → Only `main` writes the cache by default; PRs read it

DEBUGGING COMMANDS

COMMAND	SHOWS
`--stacktrace / -S`	Stack trace (full)
`--info / --debug`	More log detail
`--warning-mode all`	Every deprecation
`dependencyInsight --dependency x`	Why a version won
`buildEnvironment`	Plugin classpath
`--refresh-dependencies`	Re-download metadata
`--stop`	Kill stuck daemons

COMMON ERRORS → FIX

ERROR	USUAL FIX
`Could not resolve g:a:v`	Check repos, credentials, `--offline`
`Unsupported class file major version`	JDK mismatch — set a toolchain
`Configuration cache problems found`	Open the linked HTML report
`Timeout waiting to lock`	Another build running; `--stop`
`OutOfMemoryError: Metaspace`	Raise `org.gradle.jvmargs`
`Plugin [id: …] was not found`	Add `gradlePluginPortal()` to `pluginManagement`

⚠ COMMON MISTAKES

- Deleting `~/.gradle` at the first error — it rarely helps and costs a full re-download
- Caching `~/.gradle` by hand **and** with `setup-gradle` — double caches, slower restores
- Running CI on a different JDK than developers without a toolchain — "works on my machine"

✓ TIPS & BEST PRACTICES

- Remove just one corrupted entry: `rm -rf ~/.gradle/caches/modules-2/files-2.1/<group>`
- Read the **first** error in the output; later ones are often follow-ons
- Add `--continue` in CI to see failures from every module

QUICK SUMMARY

- `setup-gradle` — CI caching done right
- `--stacktrace --info` — first debugging step
- `dependencyInsight` — version questions
- Clear caches — last resort

Quick Reference: CLI & gradle.properties

common tasks · CLI flags · task path syntax · gradle.properties keys · wrapper commands · cache & daemon commands

COMMON TASKS

TASK	DOES
`build`	assemble + check
`assemble`	Build outputs only
`check`	Tests + verification
`test`	Unit tests
`clean`	Delete `build/`
`jar`	Package classes
`run`	Run (`application`)
`installDist`	App + start scripts
`publishToMavenLocal`	Install to `~/.m2`
`dependencies`	Dependency trees
`dependencyInsight`	Why a version
`tasks --all`	List every task
`projects`	Project tree
`javaToolchains`	Detected JDKs

COMMAND-LINE FLAGS

FLAG	EFFECT
`-x task`	Exclude a task
`--continue`	Keep going after failures
`-m / --dry-run`	Show plan only
`--rerun`	Rerun one task
`--rerun-tasks`	Rerun all tasks
`--offline`	No network
`--refresh-dependencies`	Recheck remotes
`--build-cache`	Use build cache
`--configuration-cache`	Use config cache
`--parallel`	Parallel projects
`--scan`	Build scan
`-q / -i / -d`	Log level
`-S / --stacktrace`	Stack traces
`-Pk=v / -Dk=v`	Project / system prop

TASK SELECTION

SYNTAX	RUNS
`test`	In all projects
`:test`	Root project only
`:app:test`	One project
`:lib:core:jar`	Nested project
`-p app test`	From a directory
`cTJ`	compileTestJava
`test --tests '*Foo'`	Task option
`:incl:test`	Included build

GRADLE.PROPERTIES KEYS

KEY	PURPOSE
`org.gradle.jvmargs`	Daemon JVM flags (`-Xmx4g`)
`org.gradle.parallel`	Parallel project execution
`org.gradle.caching`	Build cache on
`org.gradle.configuration-cache`	Configuration cache on
`org.gradle.workers.max`	Max parallel workers
`org.gradle.daemon.idletimeout`	Idle ms before exit
`org.gradle.java.installations.paths`	Extra JDK folders
`org.gradle.warning.mode`	`all` / `fail` / `summary` / `none`
`org.gradle.console`	`plain` / `rich` / `verbose`
`systemProp.http.proxyHost`	JVM system property

WRAPPER, DAEMON & CACHES

COMMAND	DOES
`gradle wrapper --gradle-version 9.8.0`	Create / pin wrapper
`./gradlew wrapper --gradle-version latest`	Upgrade wrapper
`./gradlew --version`	Gradle + JVM versions
`./gradlew updateDaemonJvm --jvm-version=21`	Pin daemon JVM
`./gradlew --status`	Running daemons
`./gradlew --stop`	Stop daemons
`~/.gradle/caches/`	Dependency + build cache
`~/.gradle/gradle.properties`	Per-user settings

Quick Reference: Build Script Snippets

SETTINGS.GRADLE.KTS

```
pluginManagement {
    repositories { gradlePluginPortal() }
    includeBuild("build-logic")
}
plugins {
    id("org.gradle.toolchains.foojay-resolver-convention")
        version "1.0.0"
}
dependencyResolutionManagement {
    repositories { mavenCentral() }
}
rootProject.name = "my-app"
include("app", "core")
```

BUILD.GRADLE.KTS

```
plugins {
    `java-library`
    alias(libs.plugins.spring.boot)
}
group = "com.example"
version = "1.0.0"
java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}
tasks.test { useJUnitPlatform() }
```

DEPENDENCY CONFIGURATIONS

```
dependencies {
    api(libs.jackson.databind)
    implementation(project(":core"))
    implementation(platform(libs.spring.boot.bom))
    compileOnly(libs.lombok)
    annotationProcessor(libs.lombok)
    runtimeOnly(libs.postgresql)
    testImplementation(platform(libs.junit.bom))
    testImplementation(libs.junit.jupiter)
    testRuntimeOnly(libs.junit.launcher)
}
```

GRADLE/LIBS.VERSIONS.TOML

```
[versions]
junit = "5.13.4"

[libraries]
guava = "com.google.guava:guava:33.4.0-jre"
junit-bom = { module = "org.junit:junit-bom",
              version.ref = "junit" }
junit-jupiter = {
    module = "org.junit.jupiter:junit-jupiter" }

[bundles]
testing = ["junit-jupiter"]

[plugins]
spring-boot = { id = "org.springframework.boot",
                version = "4.1.0" }
```

REGISTERING & CONFIGURING TASKS

```
val gen = tasks.register<Copy>("copyDocs") {
    from("docs"); into(layout.buildDirectory.dir("site"))
}
tasks.named("assemble") { dependsOn(gen) }
tasks.withType<JavaCompile>().configureEach {
    options.encoding = "UTF-8"
}
tasks.register("hello") {
    val v = project.version.toString()
    doLast { println("v$v") }
}
```

CUSTOM TASK CLASS

```
abstract class Stamp : DefaultTask() {
    @get:Input abstract val text: Property<String>
    @get:OutputFile abstract val out: RegularFileProperty
    @TaskAction fun run() =
        out.get().asFile.writeText(text.get())
}
tasks.register<Stamp>("stamp") {
    text = "built"
    out = layout.buildDirectory.file("stamp.txt")
}
```